

Project Title
Centralised College Database Management System
Project submitted to
SRM University – AP , Andhra Pradesh
For the partial fulfilment of the requirements to award the
degree of
Bachelor of Technology
In
Computer Science and Engineering
School of Engineering and Sciences



Submitted by
Sreeja.N-AP24110010902
Under the Guidance of
Dr. Deepthi K C Kakumani
SRM University-AP
Neerukonda,
Mangalagiri, Guntur
Andhra Pradesh
522240
[April-2026]

CERTIFICATE

This is to certify that the project entitled College Centralised Database Management System has been successfully completed by Sreeja N(AP24110010902) as a required academic component of the course CSE 209: DataBase Management Systems during Semester 4 of Academic Year 2025-26 in the Department of Computer Science and Engineering at SRM University, AP. This work executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of 19-04-2026

(Signature of the Course Instructor)
Dr. Deepthi K C Kakumani
Assistant Professor,
Department of Computer Science and Engineering,
SRM University, AP

Acknowledgement

I wish to express our sincere gratitude to all the people who supported and guided us throughout this project. We would like to thank the Prof. Ch Satish Kumar, Vice Chancellor, and Professor C.V. Tomy, Dean of School of Engineering and Sciences (SEAS), for fostering an environment of learning and innovation at SRM University AP. We are extremely thankful to our Faculty Mentor, Dr. Deepthi K C Kakumani, Assistant Professor, Department of Computer Science. Her invaluable guidance, encouragement, and insightful feedback on DataBase Management Systems were instrumental in the successful completion of this project.

TABLE OF CONTENTS

1. Identification of Project	5
2. Project Background	6
3. Description of the Project	7
4. Entity-Relationship (ER) Diagram	8
5. Description of ER Diagram	9
6. Conversion of ER Diagram into Tables	10
6.1 Department Table	10
6.2 Faculty Table	10
6.3 Student Table	11
6.4 Course Table	11
6.5 Enrollment Table	12
6.6 Attendance Table	12
6.7 Result Table	13
7. Description of Tables	14
8. Normalization up to 3NF	18
9. Creation of Tables (SQL)	20
10. Creation of Data	23
11. SQL Queries	24
12. Creation of Views	26
13. Conclusion	28
14. References	29

1. Identification of Project

Project Title: Centralised College Database Management System

The Centralised College Database Management System is designed to store and manage academic and administrative information of a college in a structured manner. The system maintains details about departments, faculty members, students, courses, enrolments, attendance records, and examination results. The database helps in organising data efficiently and reduces manual work by maintaining all information in one centralised system. The project is implemented using SQL and follows relational database concepts such as entities, relationships, primary keys, and foreign keys.

2. Project Background

In educational institutions, a large amount of information is generated every day related to students, teachers, courses, and academic performance. Traditionally, this information was stored manually in registers or separate files, which made data handling difficult and time consuming. Manual systems often lead to problems such as duplication of data, data inconsistency, and difficulty in updating records. It also becomes difficult to retrieve information quickly when required.

With the advancement of technology, database systems are used to store and manage large amounts of data efficiently. A Database Management System allows data to be stored in tables and establishes relationships between them. This helps to maintain accuracy, consistency, and security of data. The Centralised College Database Management System is designed to solve the problems of manual record keeping by providing a structured way to store academic data. It ensures that information about students, faculty, courses, attendance, and results can be accessed easily and updated without errors.

Using DBMS concepts like ER diagrams, primary keys, foreign keys, and normalisation, the system ensures proper relationships between tables and maintains data integrity. The database structure helps avoid repeated data and makes it easier to perform operations like inserting, updating, and retrieving information.

The main aim of this project is to design a structured database that simplifies college data management and demonstrates the practical use of database concepts.

3. Description of the Project

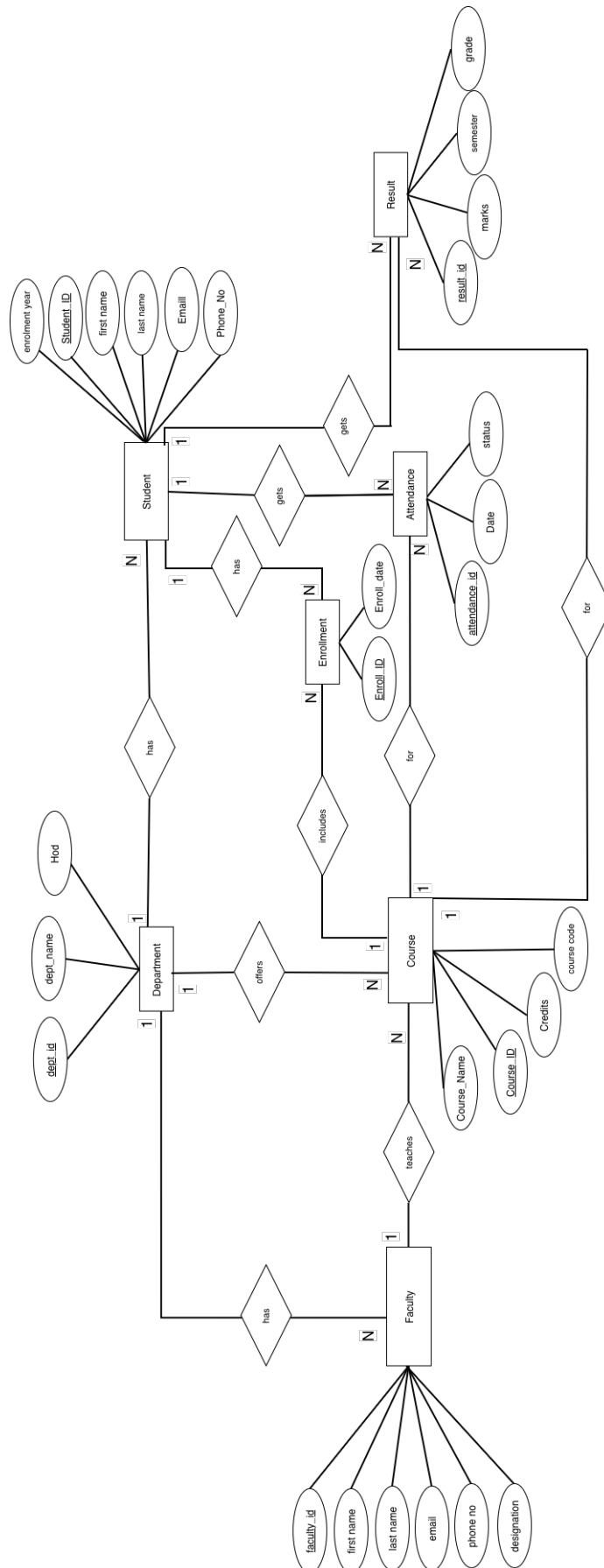
The Centralised College Database Management System is developed to manage academic information in an organised and efficient way. The system consists of multiple related tables that store different types of data required in a college. The database maintains information about departments, faculty members, students, courses offered, enrolment details, attendance records, and examination results. Each entity is connected through relationships to ensure data consistency and integrity.

The Department table stores the details of different departments available in the college along with the head of department. The Faculty table stores information about teachers including their personal details, designation, and the department they belong to. The Student table contains student information such as name, email, phone number, and enrolment year. Each student belongs to a specific department.

The Course table stores information about subjects offered by the college and identifies which faculty teaches a particular course. Students enrol in courses, and this information is stored in the Enrolment table. The Attendance table keeps track of whether students are present or absent for each course on specific dates. The Result table stores the marks and grades obtained by students in different courses and semesters.

The project demonstrates how data can be structured into tables and connected using keys. Primary keys uniquely identify each record, while foreign keys create relationships between tables. This system helps in easy retrieval of information using SQL queries and ensures that data remains accurate and consistent. The project can be further extended to include additional features such as timetable management, fee details, and library records.

4. Entity-Relationship (ER) Diagram



5. Description of ER diagram

Entities:

Student – stores student details

Department – stores department information

Faculty – stores faculty details

Course – stores subject information

Enrollment – stores course registration details

Attendance – stores attendance records

Result – stores marks and grades

Relationships

Department → Student (1:M)

One department has many students

Department → Faculty (1:M)

One department has many faculty members

Department → Course (1:M)

One department offers many courses

Student → Enrollment (1:M)

One student can enroll in many courses

Course → Enrollment (1:M)

One course can have many students

Enrollment → Attendance (1:M)

Attendance records belong to enrollments

Enrollment → Result (1:1)

Each enrollment has one result

The Enrollment entity acts as an associative entity between Student and Course.

6. Conversion of ER Diagram into Tables

The ER diagram is converted into relational tables by transforming each entity into a table and each attribute into a column.

6.1. DEPARTMENT → Department Table

The Department entity is converted into the Department table. Each department is uniquely identified using the primary key DepartmentID. The table stores the department name and the Head of Department (HOD). This table acts as a parent table for Student, Faculty, and Course tables.

DepartmentID	DepartmentName	HOD
1	Computer Science	Dr. Sharma
2	Electronics	Dr. Kumar
3	Mechanical	Dr. Reddy
4	Civil	Dr. Rao
5	biology	dr.mahesh
NULL	NULL	NULL

6.2. FACULTY → Faculty Table

The Faculty entity is converted into the Faculty table. Each faculty member is uniquely identified using FacultyID as the primary key. The DepartmentID attribute is used as a foreign key to establish the relationship between Faculty and Department. This shows that each faculty member belongs to one department.

FacultyID	FirstName	LastName	Email	Phone	DepartmentID	Designation
1	Ramesh	Kumar	ramesh@college.com	9876543201	1	Professor
2	Kavitha	Rao	kavitha@college.com	9876543202	1	Assistant Professor
3	Arun	Prasad	arun@college.com	9876543203	2	Professor
4	Meena	Reddy	meena@college.com	9876543204	3	Lecturer
NULL	NULL	NULL	NULL	NULL	NULL	NULL

6.3. STUDENT → Student Table

The Student entity is converted into the Student table. Each student is uniquely identified using StudentID as the primary key. The DepartmentID attribute acts as a foreign key linking students to their respective departments. This shows that each student belongs to a particular department.

	StudentID	FirstName	LastName	Email	Phone	DepartmentID	EnrollmentYear
	1	Sreeja	R	sreeja@gmail.com	9876543210	1	2023
	2	Rahul	Sharma	rahul@gmail.com	9876543211	1	2023
	3	Priya	Nair	priya@gmail.com	9876543212	2	2022
	4	Kiran	Rao	kiran@gmail.com	9876543213	3	2021
	5	Anjali	Das	anjali@gmail.com	9876543214	4	2022
	6	Mani	Sharma	mani@gmail.com	9847626200	1	2000
	7	man	sharma	sde@gmail.com	123456789	1	2028
	NULL	NULL	NULL	NULL	NULL	NULL	NULL

6.4. COURSE → Course Table

The Course entity is converted into the Course table. Each course is uniquely identified using CourseID as the primary key. The DepartmentID foreign key shows which department offers the course. The FacultyID foreign key indicates which faculty member teaches the course. This establishes relationships between Course, Department, and Faculty.

	CourseID	CourseName	CourseCode	Credits	DepartmentID	FacultyID
	1	Database Management System	DBMS101	4	1	1
	2	Data Structures	DS102	3	1	2
	3	Digital Electronics	DE201	4	2	3
	4	Thermodynamics	ME301	3	3	4
	5	Structural Analysis	CE401	4	4	4
	NULL	NULL	NULL	NULL	NULL	NULL

6.5. ENROLLMENT → Enrollment Table

The relationship between Student and Course is many-to-many because one student can enroll in multiple courses and one course can have many students. To resolve this relationship, the Enrollment entity is converted into the Enrollment table. The primary key is EnrollmentID. The table includes StudentID and CourseID as foreign keys to link students with courses.

EnrollmentID	EnrollmentDate	StudentID	CourseID
1	2026-01-10	1	1
2	2026-01-11	1	2
3	2026-01-12	2	1
4	2026-01-13	3	3
5	2026-01-14	4	4
6	2026-01-15	5	5
7	2026-04-09	4	1
NULL	NULL	NULL	NULL

6.6. ATTENDANCE → Attendance Table

The Attendance entity is converted into the Attendance table. Each attendance record is uniquely identified using AttendanceID as the primary key. The table stores attendance status and date. The StudentID and CourseID attributes act as foreign keys linking attendance records to the respective student and course.

AttendanceID	Date	Status	StudentID	CourseID
1	2026-02-01	Present	1	1
2	2026-02-01	Absent	1	2
3	2026-02-01	Present	2	1
4	2026-02-02	Present	3	3
5	2026-02-02	Absent	4	4
6	2026-02-03	Present	5	5
NULL	NULL	NULL	NULL	NULL

6.7. RESULT → Result Table

The Result entity is converted into the Result table. Each result record is uniquely identified using ResultID as the primary key. The table stores marks, grade, and semester information. The StudentID foreign key links the result to the student, and the CourseID foreign key links the result to the course. This allows tracking academic performance of students for each course.

ResultID	Marks	Grade	Semester	StudentID	CourseID	
1	88	A	1	1	1	
2	76	B	1	1	2	
3	91	A+	1	2	1	
4	69	B	2	3	3	
5	72	B+	2	4	4	
6	85	A	2	5	5	
NULL	NULL	NULL	NULL	NULL	NULL	

7. Description of Tables

Below is the detailed schema outlining the exact structure, data types, and constraints of the tables derived from the ER Diagram.

7.1. DEPARTMENT

Stores information about academic departments available in the institution.

DepartmentID (PK): INT, Auto-increment.

Unique identifier for each department.

DepartmentName: VARCHAR, Name of the department (e.g., Computer Science, Electronics).

HOD: VARCHAR, Name of the Head of Department responsible for managing the department.

7.2. FACULTY

Stores professional and contact details of teaching staff working in the institution.

FacultyID (PK): INT, Auto-increment.

Unique identifier for each faculty member.

FirstName: VARCHAR, Faculty member's given name.

LastName: VARCHAR, Faculty member's surname.

Email: VARCHAR, Official email address of the faculty member.

Phone: VARCHAR, Contact telephone number.

DepartmentID (FK): INT, References DEPARTMENT(DepartmentID).

Indicates the department to which the faculty belongs.

Designation: VARCHAR, Academic role or job title (e.g., Professor, Lecturer).

7.3. STUDENT

Stores academic and personal information of students enrolled in the college.

StudentID (PK): INT, Auto-increment.

Unique identifier for each student.

FirstName: VARCHAR, Student's given name.

LastName: VARCHAR, Student's surname.

Email: VARCHAR, Student's email address.

Phone: VARCHAR, Student's contact number.

DepartmentID (FK): INT, References DEPARTMENT(DepartmentID).

Indicates the department in which the student is enrolled.

EnrollmentYear: INT, Year in which the student joined the institution.

7.4. COURSE

Stores details of academic subjects offered by the departments.

CourseID (PK): INT, Auto-increment.

Unique identifier for each course.

CourseName: VARCHAR, Name of the subject (e.g., Database Management System).

CourseCode: VARCHAR, Unique code assigned to the course.

Credits: INT, Credit value assigned to the course.

DepartmentID (FK): INT, References DEPARTMENT(DepartmentID).

Indicates which department offers the course.

FacultyID (FK): INT, References FACULTY(FacultyID).

Indicates which faculty teaches the course.

7.5. ENROLLMENT

Acts as a junction table that records which students are registered for which courses.

EnrollmentID (PK): INT, Auto-increment.

Unique identifier for each enrollment record.

EnrollmentDate: DATE, Date on which the student enrolled in the course.

StudentID (FK): INT, References STUDENT(StudentID).

Identifies the student who enrolled in the course.

CourseID (FK): INT, References COURSE(CourseID).

Identifies the course in which the student enrolled.

7.6. ATTENDANCE

Stores attendance records of students for each course session.

AttendanceID (PK): INT, Auto-increment.

Unique identifier for each attendance record.

Date: DATE, Date on which attendance is recorded.

Status: VARCHAR, Attendance state (e.g., Present, Absent).

StudentID (FK): INT, References STUDENT(StudentID).

Indicates which student's attendance is recorded.

CourseID (FK): INT, References COURSE(CourseID).

Indicates for which course attendance is recorded.

7.7. RESULT

Stores academic performance details of students in different courses and semesters.

ResultID (PK): INT, Auto-increment.

Unique identifier for each result entry.

Marks: INT, Numerical score obtained by the student.

Grade: VARCHAR, Grade assigned based on marks (e.g., A, B+).

Semester: INT, Academic semester in which the result was achieved.

StudentID (FK): INT, References STUDENT(StudentID).

Indicates the student to whom the result belongs.

CourseID (FK): INT, References COURSE(CourseID).

Indicates the course for which the result is recorded.

8. Normalisation up to 3NF

Normalization is the process of organizing data in a database to reduce redundancy and improve consistency. It helps maintain accurate relationships between tables and avoids anomalies. In the Centralized College Database System, normalization is applied up to Third Normal Form (3NF) to ensure efficient data storage and better database design.

First Normal Form (1NF)

A table is in First Normal Form if:

- Each column contains only single (atomic) values
- Each record is unique
- There are no repeating groups
- Each table has a primary key

In this project:

- Student table stores single values for name, email, and phone
- Course table stores one course per row
- Each table has a unique primary key

Hence, all tables satisfy 1NF.

Second Normal Form (2NF)

A table is in Second Normal Form if:

- It is already in 1NF
- All non-key attributes fully depend on the primary key
- There is no partial dependency

In this project:

- Each table has a single primary key
- Attributes depend completely on the primary key
- Enrollment details depend on EnrollmentID

Hence, all tables satisfy 2NF.

Third Normal Form (3NF)

A table is in Third Normal Form if:

- It is already in 2NF
- There is no transitive dependency
- Non-key attributes depend only on the primary key

In this project:

- Department details stored in Department table
- Student, Faculty, and Course stored separately
- Attendance and Result stored in separate tables

Hence, the database satisfies 3NF.

9. Creation of Tables (SQL)

The following SQL statements are used to create the tables required for the Centralized College Database System. These tables store information related to students, departments, faculty, courses, enrollments, attendance, and results.

Each table is created with appropriate primary keys and foreign keys to maintain relationships and ensure data consistency.

Creating DEPARTMENT Table

The **DEPARTMENT** table stores details of different departments available in the college.

```
• ○ CREATE TABLE Department (  
    DepartmentID INT AUTO_INCREMENT PRIMARY KEY,  
    DepartmentName VARCHAR(100) NOT NULL,  
    HOD VARCHAR(100)  
);
```

Creating STUDENT Table

```
• ○ CREATE TABLE Student (  
    StudentID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Email VARCHAR(100),  
    Phone VARCHAR(15),  
    DepartmentID INT NOT NULL,  
    EnrollmentYear INT NOT NULL,  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)  
);
```

The **STUDENT** table stores personal and academic information of students.

Creating FACULTY Table

The **FACULTY** table stores information about faculty members working in different departments.

```
• CREATE TABLE Faculty (  
    FacultyID INT AUTO_INCREMENT PRIMARY KEY,  
    FirstName VARCHAR(50) NOT NULL,  
    LastName VARCHAR(50) NOT NULL,  
    Email VARCHAR(100),  
    Phone VARCHAR(15),  
    DepartmentID INT NOT NULL,  
    Designation VARCHAR(50) NOT NULL,  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)  
);
```

Creating COURSE Table

The **COURSE** table stores details of subjects offered by various departments.

```
CREATE TABLE Course (  
    CourseID INT AUTO_INCREMENT PRIMARY KEY,  
    CourseName VARCHAR(100) NOT NULL,  
    CourseCode VARCHAR(20) NOT NULL,  
    Credits INT NOT NULL,  
    DepartmentID INT NOT NULL,  
    FacultyID INT NOT NULL,  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),  
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)  
);
```

Creating ENROLLMENT Table

The **ENROLLMENT** table records which students are registered for which courses.

```
• CREATE TABLE Enrollment (  
    EnrollmentID INT AUTO_INCREMENT PRIMARY KEY,  
    EnrollmentDate DATE NOT NULL,  
    StudentID INT NOT NULL,  
    CourseID INT NOT NULL,  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

Creating ATTENDANCE Table

The **ATTENDANCE** table stores attendance percentage of students for each course.

```
CREATE TABLE Attendance (  
    AttendanceID INT AUTO_INCREMENT PRIMARY KEY,  
    AttendanceDate DATE NOT NULL,  
    Status VARCHAR(20) NOT NULL,  
    StudentID INT NOT NULL,  
    CourseID INT NOT NULL,  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

Creating RESULT Table

The **RESULT** table stores marks and grades obtained by students in each course.

```
CREATE TABLE Result (  
    ResultID INT AUTO_INCREMENT PRIMARY KEY,  
    Marks INT NOT NULL,  
    Grade VARCHAR(5) NOT NULL,  
    Semester INT NOT NULL,  
    StudentID INT NOT NULL,  
    CourseID INT NOT NULL,  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

Creating USER Table

The **USER** table stores login credentials and role-based access details of users such as admin, faculty, and students.

```
CREATE TABLE Users (  
    UserID INT AUTO_INCREMENT PRIMARY KEY,  
    Username VARCHAR(50) NOT NULL UNIQUE,  
    Password VARCHAR(50) NOT NULL,  
    Role VARCHAR(20) NOT NULL,  
    LinkedID INT  
);
```

10. Creation of Data

Sample records are inserted to test functionality such as student enrollment,

```
INSERT INTO Department (DepartmentName, HOD) VALUES
```

```
('Computer Science and Engineering', 'Dr. Sharma'),
('Electronics and Communication Engineering', 'Dr. Kumar'),
('Mechanical Engineering', 'Dr. Reddy'),
('Civil Engineering', 'Dr. Rao');
```

```
INSERT INTO Faculty (FirstName, LastName, Email, Phone, DepartmentID, Designation) VALUES
```

```
('Ramesh', 'Kumar', 'ramesh@college.com', '9876543201', 1, 'Professor'),
('Kavitha', 'Rao', 'kavitha@college.com', '9876543202', 1, 'Assistant Professor'),
('Arun', 'Prasad', 'arun@college.com', '9876543203', 2, 'Professor'),
('Meena', 'Reddy', 'meena@college.com', '9876543204', 3, 'Lecturer');
```

```
INSERT INTO Student (FirstName, LastName, Email, Phone, DepartmentID, EnrollmentYear) VALUES
```

```
('Sreeja', 'N', 'sreeja@gmail.com', '9876543210', 1, 2023),
('Rahul', 'Sharma', 'rahul@gmail.com', '9876543211', 1, 2023),
('Priya', 'Nair', 'priya@gmail.com', '9876543212', 2, 2022),
('Kiran', 'Rao', 'kiran@gmail.com', '9876543213', 3, 2021),
('Anjali', 'Das', 'anjali@gmail.com', '9876543214', 4, 2022);
```

```
INSERT INTO Course (CourseName, CourseCode, Credits, DepartmentID, FacultyID) VALUES
```

```
('Database Management Systems', 'DBMS101', 4, 1, 1),
('Data Structures', 'DS102', 3, 1, 2),
('Digital Electronics', 'DE201', 4, 2, 3),
('Thermodynamics', 'ME301', 3, 3, 4),
('Structural Analysis', 'CE401', 4, 4, 4);
```

```
INSERT INTO Enrollment (EnrollmentDate, StudentID, CourseID) VALUES
```

```
('2026-01-10', 1, 1),
('2026-01-11', 1, 2),
('2026-01-12', 2, 1),
('2026-01-13', 3, 3),
('2026-01-14', 4, 4),
('2026-01-15', 5, 5);
```

```
INSERT INTO Attendance (AttendanceDate, Status, StudentID, CourseID) VALUES
```

```
('2026-02-01', 'Present', 1, 1),
('2026-02-01', 'Absent', 1, 2),
('2026-02-01', 'Present', 2, 1),
('2026-02-02', 'Present', 3, 3),
('2026-02-02', 'Absent', 4, 4),
('2026-02-03', 'Present', 5, 5);
```

```
INSERT INTO Result (Marks, Grade, Semester, StudentID, CourseID) VALUES
```

```
(88, 'A', 1, 1, 1),
(76, 'B', 1, 1, 2),
(91, 'A+', 1, 2, 1),
(69, 'B', 2, 3, 3),
(72, 'B+', 2, 4, 4),
(85, 'A', 2, 5, 5);
```

```
INSERT INTO Users (Username, Password, Role, LinkedID) VALUES
```

```
('admin', 'admin123', 'admin', NULL),
('student1', 'student123', 'student', 1),
('student2', 'student123', 'student', 2),
('faculty1', 'faculty123', 'faculty', 1),
('faculty2', 'faculty123', 'faculty', 2);
```

attendance tracking, and result calculation

11.SQL Queries (Subqueries, Aggregate Functions, Joins)

The following SQL queries are used to retrieve meaningful information from the Centralized College Database System. These queries help in combining data from multiple tables, performing calculations, and filtering records based on specific conditions.

JOIN QUERIES

These queries combine data from multiple tables using common attributes such as StudentID and DepartmentID.

```
SELECT s.StudentID, s.FirstName, s.LastName, d.DepartmentName
FROM Student s
JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

```
SELECT s.FirstName, s.LastName, c.CourseName, e.EnrollmentDate
FROM Enrollment e
JOIN Student s ON e.StudentID = s.StudentID
JOIN Course c ON e.CourseID = c.CourseID;
```

```
SELECT s.FirstName, s.LastName, c.CourseName, r.Marks, r.Grade
FROM Result r
JOIN Student s ON r.StudentID = s.StudentID
JOIN Course c ON r.CourseID = c.CourseID;
```

AGGREGATE FUNCTION QUERIES

These queries use aggregate functions like COUNT and AVG to summarize data such as total number of students and average marks.

```
SELECT d.DepartmentName, COUNT(s.StudentID) AS TotalStudents
FROM Department d
LEFT JOIN Student s ON d.DepartmentID = s.DepartmentID
GROUP BY d.DepartmentName;
```



```
SELECT c.CourseName, AVG(r.Marks) AS AverageMarks
FROM Result r
JOIN Course c ON r.CourseID = c.CourseID
GROUP BY c.CourseName;
```

SUBQUERIES

Subqueries are used to retrieve data based on the result of another query, such as finding students scoring above average marks.

```
SELECT *
FROM Result
WHERE Marks > (SELECT AVG(Marks) FROM Result);
```

```
SELECT CourseName
FROM Course
WHERE DepartmentID = (
    SELECT DepartmentID
    FROM Department
    WHERE DepartmentName = 'Computer Science and Engineering'
);
```

12. Creation of Views

Views are virtual tables created using SQL queries to simplify complex data retrieval and improve readability.

Creating StudentDepartmentView

This view displays student details along with their department names.

```
CREATE VIEW StudentDepartmentView AS
```

```
SELECT s.StudentID, s.FirstName, s.LastName, s.Email, d.DepartmentName
```

```
FROM Student s
```

```
JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

	StudentID	FirstName	LastName	Email	DepartmentName
	1	Sreeja	N	sreeja@gmail.com	Computer Science and Engineering
	2	Rahul	Sharma	rahul@gmail.com	Computer Science and Engineering
	3	Priya	Nair	priya@gmail.com	Electronics and Communication Engineering
	4	Kiran	Rao	kiran@gmail.com	Mechanical Engineering
	5	Anjali	Das	anjali@gmail.com	Civil Engineering

Creating StudentResultView

This view shows student marks and grades for each course.

```
CREATE VIEW StudentResultView AS
```

```
SELECT s.StudentID, s.FirstName, s.LastName, c.CourseName, r.Marks, r.Grade,  
r.Semester
```

```
FROM Result r
```

```
JOIN Student s ON r.StudentID = s.StudentID
```

```
JOIN Course c ON r.CourseID = c.CourseID;
```

	StudentID	FirstName	LastName	CourseName	Marks	Grade	Semester
	1	Sreeja	N	Database Management Systems	88	A	1
	1	Sreeja	N	Data Structures	76	B	1
	2	Rahul	Sharma	Database Management Systems	91	A+	1
	3	Priya	Nair	Digital Electronics	69	B	2
	4	Kiran	Rao	Thermodynamics	72	B+	2
	5	Anjali	Das	Structural Analysis	85	A	2

Creating EnrollmentView

This view displays enrollment details of students for different courses.

```
CREATE VIEW EnrollmentView AS
```

```
SELECT e.EnrollmentID, s.FirstName, s.LastName, c.CourseName,
e.EnrollmentDate
```

```
FROM Enrollment e
```

```
JOIN Student s ON e.StudentID = s.StudentID
```

```
JOIN Course c ON e.CourseID = c.CourseID;
```

	EnrollmentID	FirstName	LastName	CourseName	EnrollmentDate
	1	Sreeja	N	Database Management Systems	2026-01-10
	2	Sreeja	N	Data Structures	2026-01-11
	3	Rahul	Sharma	Database Management Systems	2026-01-12
	4	Priya	Nair	Digital Electronics	2026-01-13
	5	Kiran	Rao	Thermodynamics	2026-01-14
	6	Anjali	Das	Structural Analysis	2026-01-15

13. Conclusion

The Centralized College Database System was successfully designed and implemented using the concepts of Database Management Systems. This project provides an efficient way to store, organize, and manage important academic information such as student details, department data, faculty records, courses, enrollments, attendance, and results. The use of a centralized database helps in reducing manual work and ensures that data is stored in a structured and reliable manner.

The database was designed using ER modeling, which clearly shows the relationships between different entities. These entities were converted into relational tables using primary keys and foreign keys to maintain data integrity. Normalization up to Third Normal Form (3NF) was applied to remove redundancy and avoid anomalies such as insertion, deletion, and update problems. This improves the accuracy and consistency of data.

SQL queries such as joins, aggregate functions, and subqueries were used to retrieve useful information from multiple tables. Views were also created to simplify complex queries and make data retrieval easier. These operations demonstrate how database techniques can be used to efficiently analyze and manage large amounts of information.

Overall, this project shows how DBMS concepts can be applied in real-life scenarios to manage college data effectively. The system is flexible, reliable, and can be expanded further by adding more features such as user authentication, report generation, and graphical dashboards. This project helps in understanding how databases are used in educational institutions to maintain organized and consistent information.

14. References

1. Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems*, Pearson Education.
2. Oracle MySQL Documentation, *MySQL Reference Manual*.
<https://dev.mysql.com/doc/>
3. W3Schools, *SQL Tutorial*.
<https://www.w3schools.com/sql/>
4. Draw.io (Diagrams.net), *ER Diagram Tool*.
<https://app.diagrams.net/>